End-to-End CI/CD Pipeline for a Calculator REST API

Nirbhay Sharma · Roll No.: **IMT2023103** · M.Tech (CSE), Term I 2026-27

GitHub: https://github.com/NirbhaySharma504/calculator_devops
Docker Hub: https://hub.docker.com/r/hades504/calculator-api

1. Objective and Technology Stack

The goal was to build a small, production-style Calculator REST API and wrap it in a fully automated DevOps pipeline: merging a change on GitHub builds, tests, containerises, publishes and redeploys the service with no manual steps. The application is deliberately simple; the work is in the pipeline around it.

Concern	Tool / choice
Application	Java 21, Spring Boot 4.1 (Spring Web, Actuator, Validation), Maven
Testing	JUnit 5 unit tests + Spring MockMvc integration tests (57 tests), Surefire XML reports
Version control	Git + GitHub: 7 feature branches, each merged through a pull request
CI/CD	Jenkins LTS (declarative Jenkinsfile), GitHub webhook through an ngrok tunnel, Poll SCM fallback, Email Extension
Containers	Docker multi-stage build; image published as hades504/calculator-api on Docker Hub
Deployment	Docker Compose: Nginx reverse proxy + API on a private bridge network

2. Pipeline Architecture

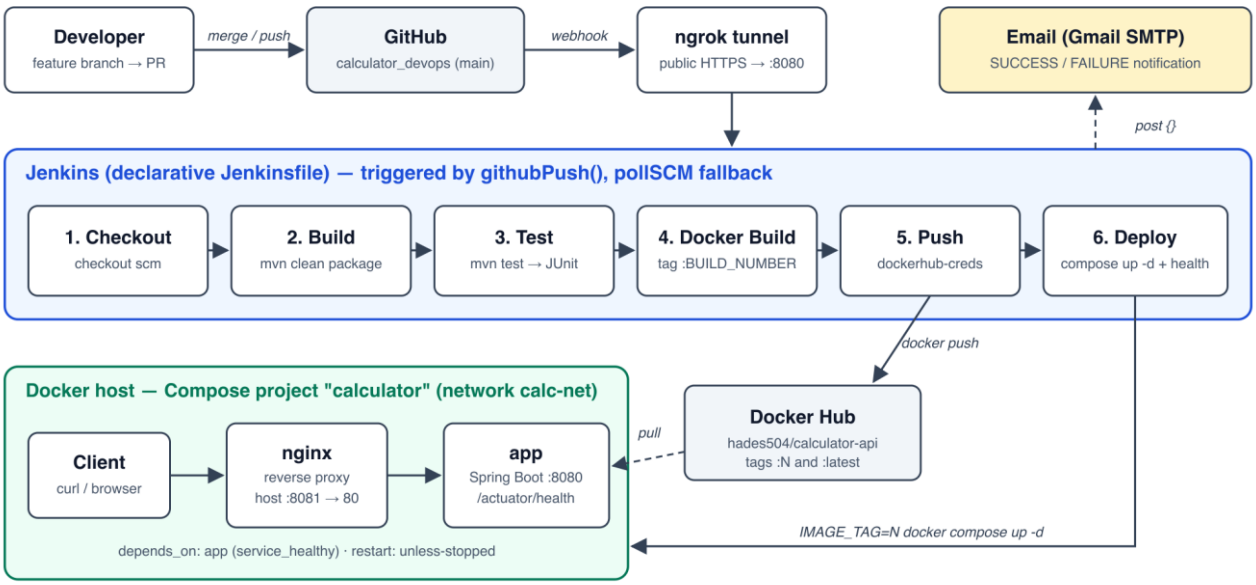


Figure 1: CI/CD pipeline. A merge to main reaches Jenkins through a GitHub webhook; Jenkins builds, tests, publishes and deploys the stack.

3. Application and Tests

The service exposes `GET /api/v1/{add|subtract|multiply|divide|modulo|power|sqrt}?a=&b=` and returns JSON such as `{"operation":"add","a":2.0,"b":3.0,"result":5.0}`. Spring Boot Actuator provides `/actuator/health` (used by the Docker, Compose and Jenkins health checks) and `/actuator/info` (shows the deployed version).

Validation lives in `CalculatorService`. A `@RestControllerAdvice` maps every invalid request to **HTTP 400** with a structured body `{timestamp, status, error, message, path}`. That covers division or modulo by zero, square root of a negative number, missing or non-numeric parameters, NaN/Infinity inputs and results that overflow.

Parameterised JUnit 5 unit tests cover every operation and edge case. Integration tests start the full Spring context and send real requests through MockMvc, checking successful responses, error bodies and the health endpoint. Maven Surefire writes JUnit XML, which Jenkins publishes on every build (57 tests, 0 failures; Figure A3).

4. Jenkins Pipeline — Stage by Stage

The pipeline is a declarative `Jenkinsfile` in the repository. `githubPush()` starts it from the GitHub webhook, and `pollSCM('H/5 * * * *')` is a fallback. `disableConcurrentBuilds()` stops two merges from deploying at once.

- **Checkout:** checkout scm fetches the exact commit that triggered the build and logs its hash, author and message.
- **Build:** mvn -B clean package -DskipTests compiles the code and produces the executable JAR.
- **Test:** mvn -B test runs all 57 tests. post { always { junit … } } publishes the results even on failure. A failing test fails the build, so nothing later runs and the live deployment is untouched.
- **Docker Build:** builds the image with two tags. :BUILD_NUMBER is immutable and traceable; :latest is convenient. An OCI label records the Git commit.
- **Push:** withCredentials injects the Docker Hub username and access token from the Jenkins Credentials store (ID dockerhub-creds). The token goes in via --password-stdin, is masked in the log and is never stored in Git.
- **Deploy:** IMAGE_TAG=<build> docker compose up -d --remove-orphans, then a loop polls /actuator/health through Nginx for up to 60 s. Smoke tests follow: add must return 5.0 and divide-by-zero must return 400 (Figure A4). On failure the stage prints the container status and logs.
- **Post actions:** emailext sends a SUCCESS or FAILURE email (the failure email attaches the log), then dangling images and old build tags are pruned locally.

Idempotency: compose up -d reconciles the running stack to the desired state and only recreates containers whose image or configuration changed. The pipeline never runs down. Re-running build #5 by hand replaced only the app container; Nginx kept running and kept serving traffic.

5. Containerisation and Deployment

- **Multi-stage Dockerfile:** a maven:3.9-eclipse-temurin-21 stage builds the JAR. The runtime stage is eclipse-temurin:21-jre-alpine and receives only the JAR, so no JDK, Maven or source code ships (90 MB compressed on Docker Hub).
- **Best practices:** pom.xml is copied and dependencies resolved before the sources, giving a cached dependency layer. The app runs as a non-root spring user, with an explicit EXPOSE 8080, a HEALTHCHECK on Actuator and JVM tuning through JAVA_TOOL_OPTIONS. A .dockerignore keeps target/, .git, IDE files and .env out of the build context.
- **Compose stack:** two services on the user-defined bridge network calc-net. app has no published port. nginx is the single entry point (host 8081, because Jenkins uses 8080). Both have health checks, restart: unless-stopped, environment variables and capped json-file logs. nginx starts only once app reports healthy (depends_on: condition: service_healthy).
- **Redeploy-safe proxy:** Nginx uses Docker's DNS (resolver 127.0.0.11) with a variable upstream, so it follows the new container IP after Jenkins recreates the app instead of returning 502.

6. Git Workflow

Work was done in small commits on feature branches (feature/calculator-api, feature/docker, feature/compose-nginx, feature/jenkins-pipeline, docs/readme-report, fix/dockerhub-namespace, release/1.0.0). Each branch was merged into main through a GitHub pull request with a merge commit, so the history stays visible (Figures A6 and A7).
The .gitignore excludes target/, IDE files, .env and key files. No credentials are in the repository: the Docker Hub token and the Gmail app password exist only in the Jenkins Credentials store, and the notification address is a Jenkins global variable.

7. Results

Build	Trigger	Outcome
#1	Manual (first run)	Failed at Docker Build: Docker Desktop was not running. Failure email sent.
#2	Manual	Failed at Push: Docker Hub rejected the login (wrong account name in the image tag). Deploy skipped; failure email sent.
#3	GitHub push (PR #6)	All stages green; image :3 pushed and deployed.
#4	GitHub push (PR #7)	All stages green; version 1.0.0 deployed automatically, visible at /actuator/info.
#5	Manual re-run	All stages green; only the app container was recreated, showing the pipeline is re-runnable.

8. Challenges Faced and Solutions

Challenge	Solution
Build #1 failed: "failed to connect to the docker API" after the Mac restarted	Docker Desktop does not start on its own; started it and re-ran. Documented as a pre-demo check.
Build #2 failed at Push: "unauthorized: incorrect username or password"	The image was tagged for a Docker Hub account that did not exist. Renamed the image to hades504/calculator-api in a fix branch and PR #6, whose merge triggered the first green build.
Homebrew-launched Jenkins starts with a minimal PATH	The Jenkinsfile prepends /opt/homebrew/bin and /usr/local/bin so mvn, docker and the Docker credential helper are found.
GitHub webhooks cannot reach localhost:8080	Exposed Jenkins through an ngrok static domain and pointed the webhook at /github-webhook/ (all deliveries HTTP 200). Jenkins login stays enforced; Poll SCM is kept as a fallback.
Port conflict: Jenkins already uses 8080	The API is not published at all; only Nginx is, on host port 8081.
Nginx returned 502 after the app container was recreated (stale IP)	Used resolver 127.0.0.11 with a variable proxy_pass so the app name is re-resolved.
Gmail refuses SMTP logins with the account password	Enabled 2-step verification and used a Google App Password over SSL on port 465, stored as a Jenkins credential.
Spring Boot 4 moved the MockMvc test annotations	Used the new spring-boot-starter-webmvc-test module and its package.

9. Conclusion

Merging a pull request now takes a change from commit to a health-checked, running deployment automatically, with published test results, a versioned image on Docker Hub and an email for every build. The capstone will extend the same repository with Ansible for configuration management, Kubernetes in place of Compose, and Prometheus and Grafana scraping the Actuator endpoints that already exist.

Appendix: Screenshots

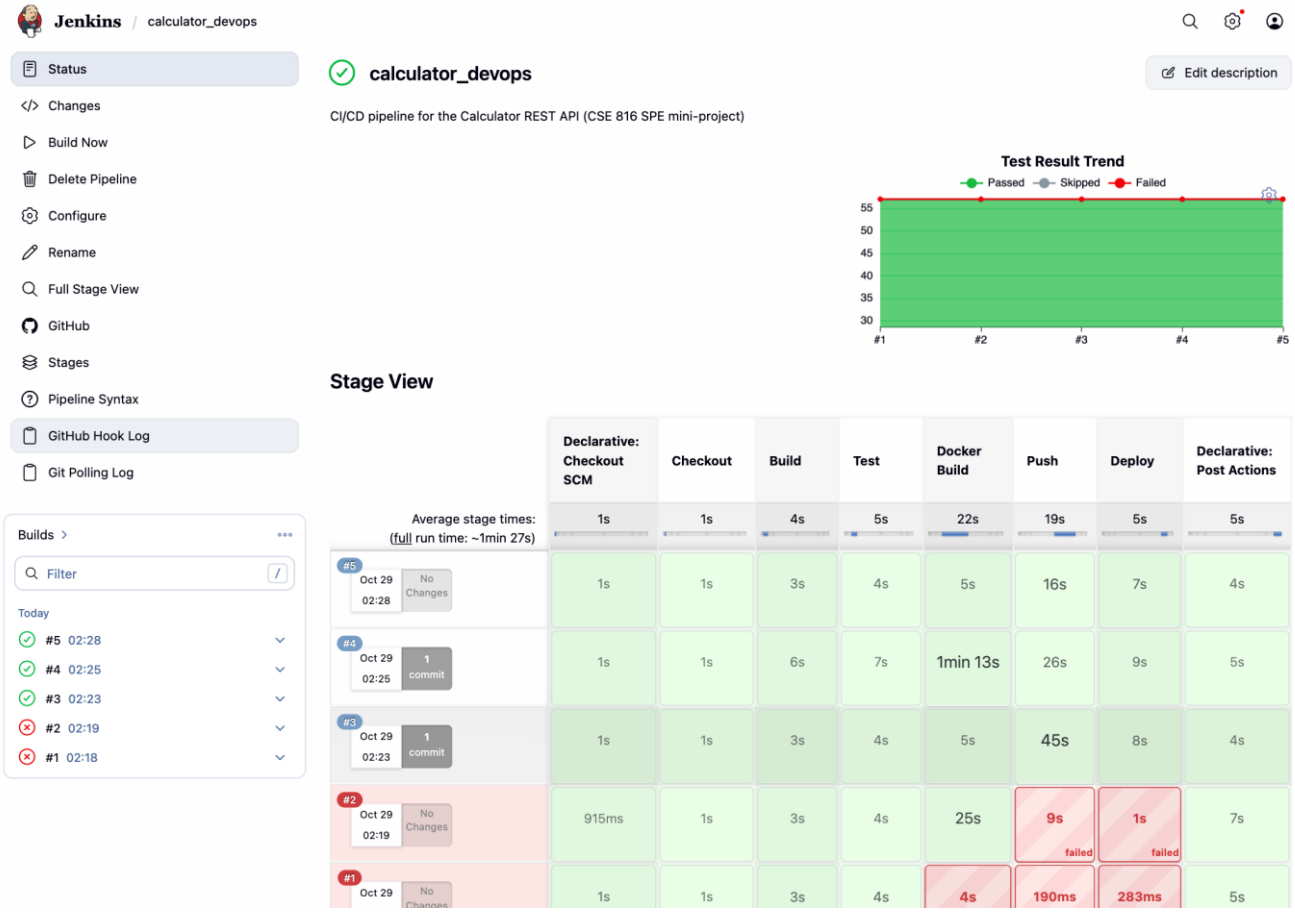


Figure A1: Jenkins Stage View for builds #1–#5 and the test result trend

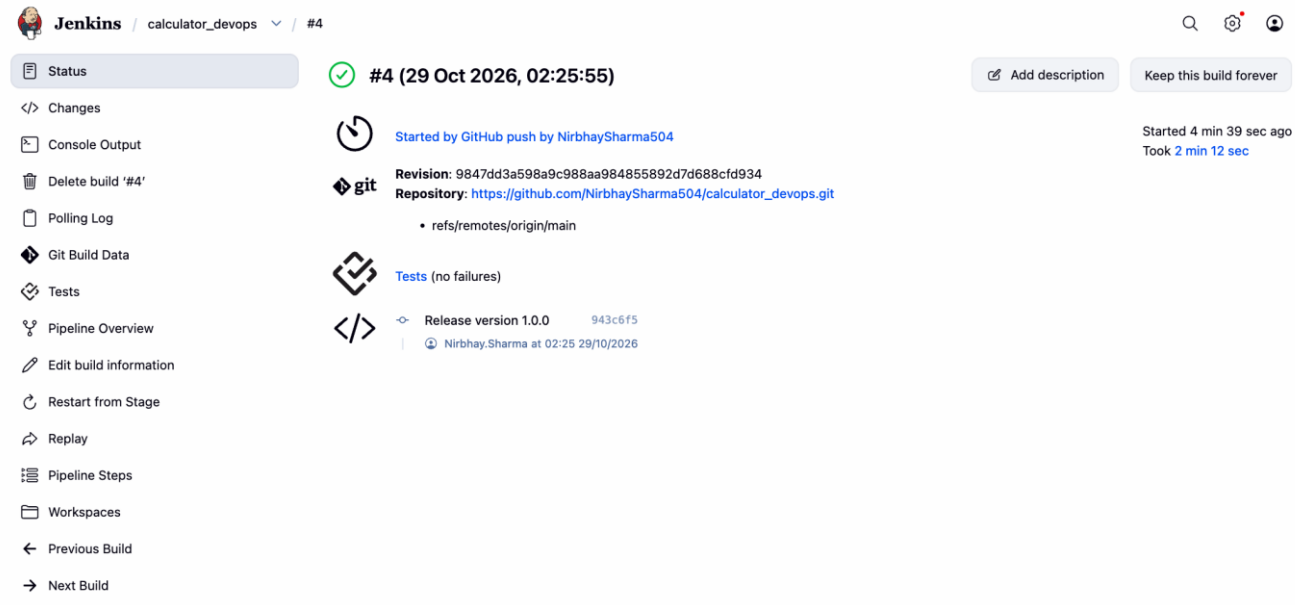


Figure A2: Build #4, started by a GitHub push (merge of PR #7, Release 1.0.0)

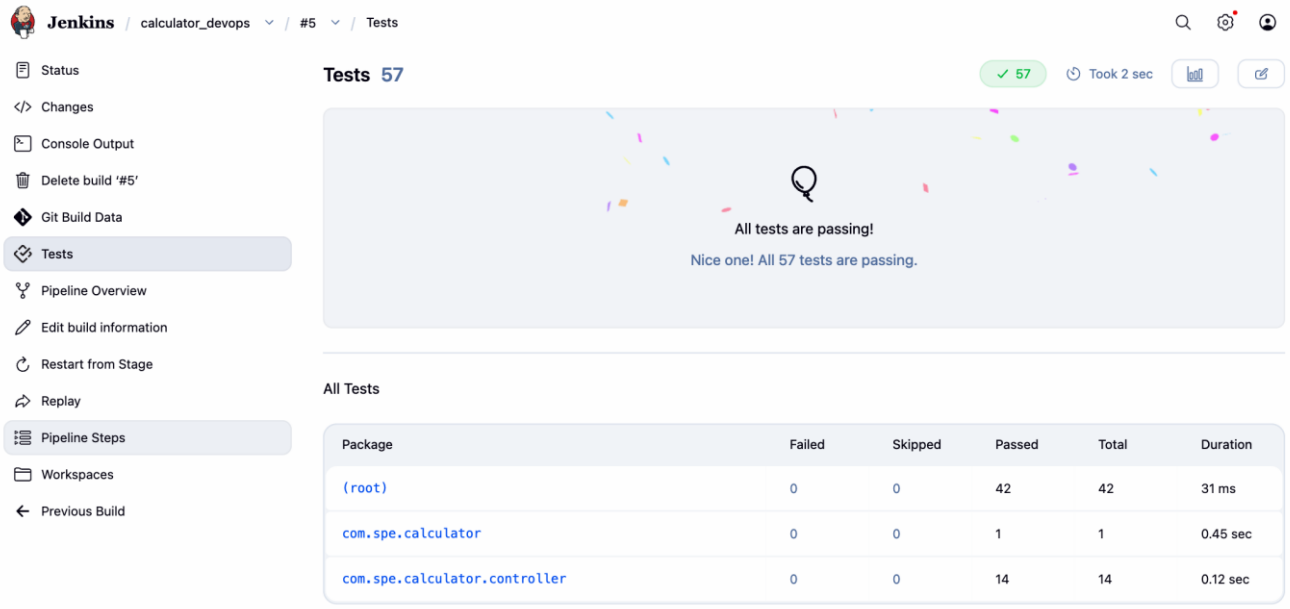


Figure A3: Jenkins test report: 57 tests, 0 failures

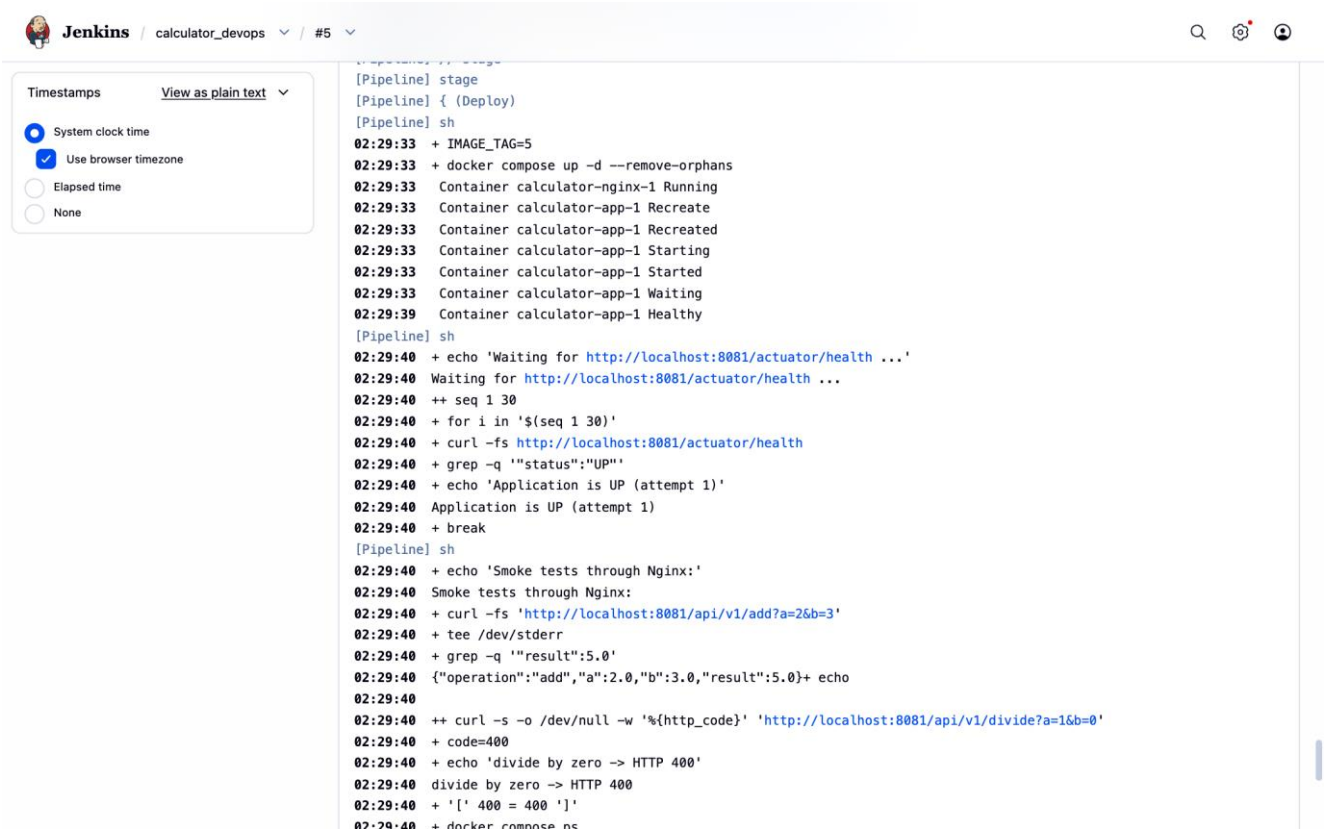


Figure A4: Deploy stage log: compose recreates only the app, health check UP, smoke tests pass

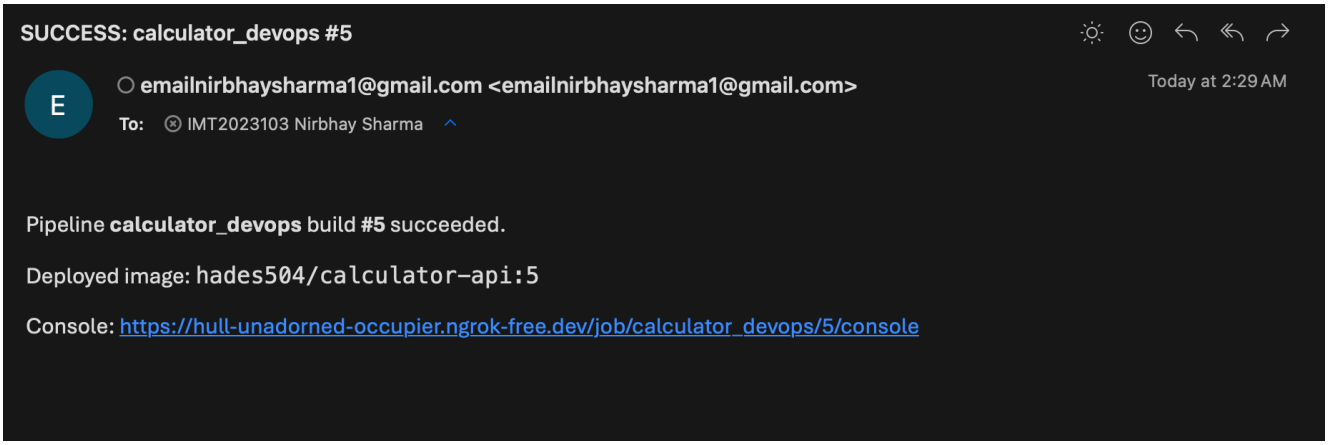


Figure A5: Build notification email from Jenkins

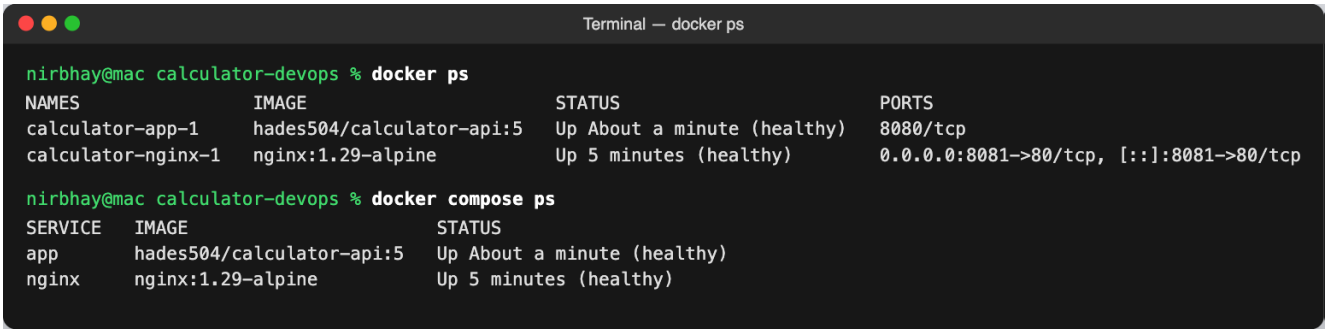


Figure A6: docker ps / docker compose ps: both containers healthy, app running build #5

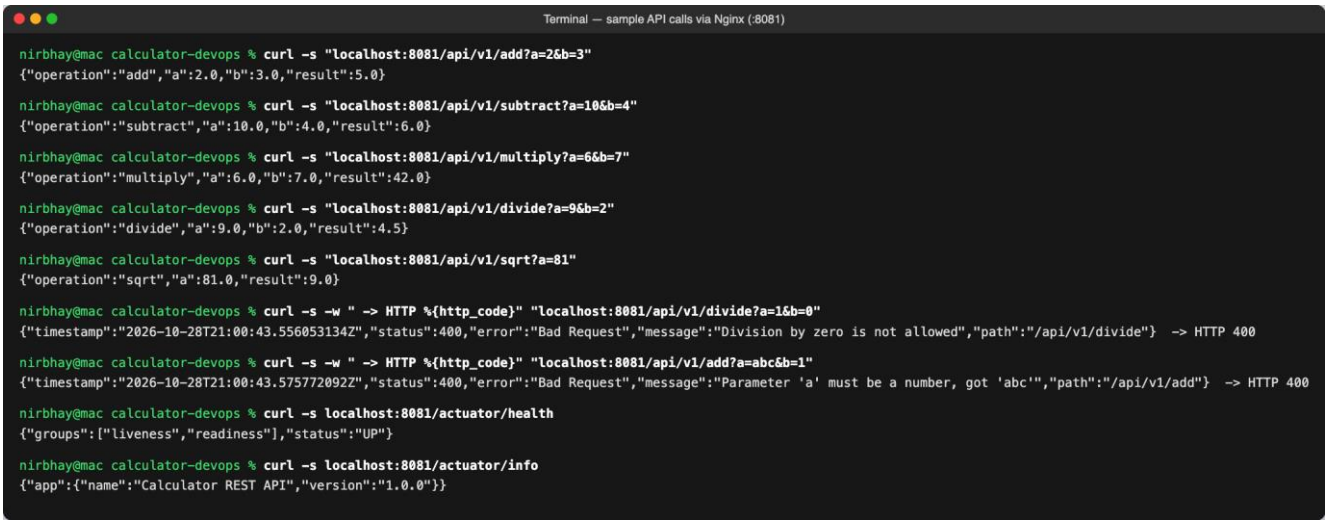


Figure A7: Sample API responses through Nginx, including HTTP 400 error bodies and version 1.0.0

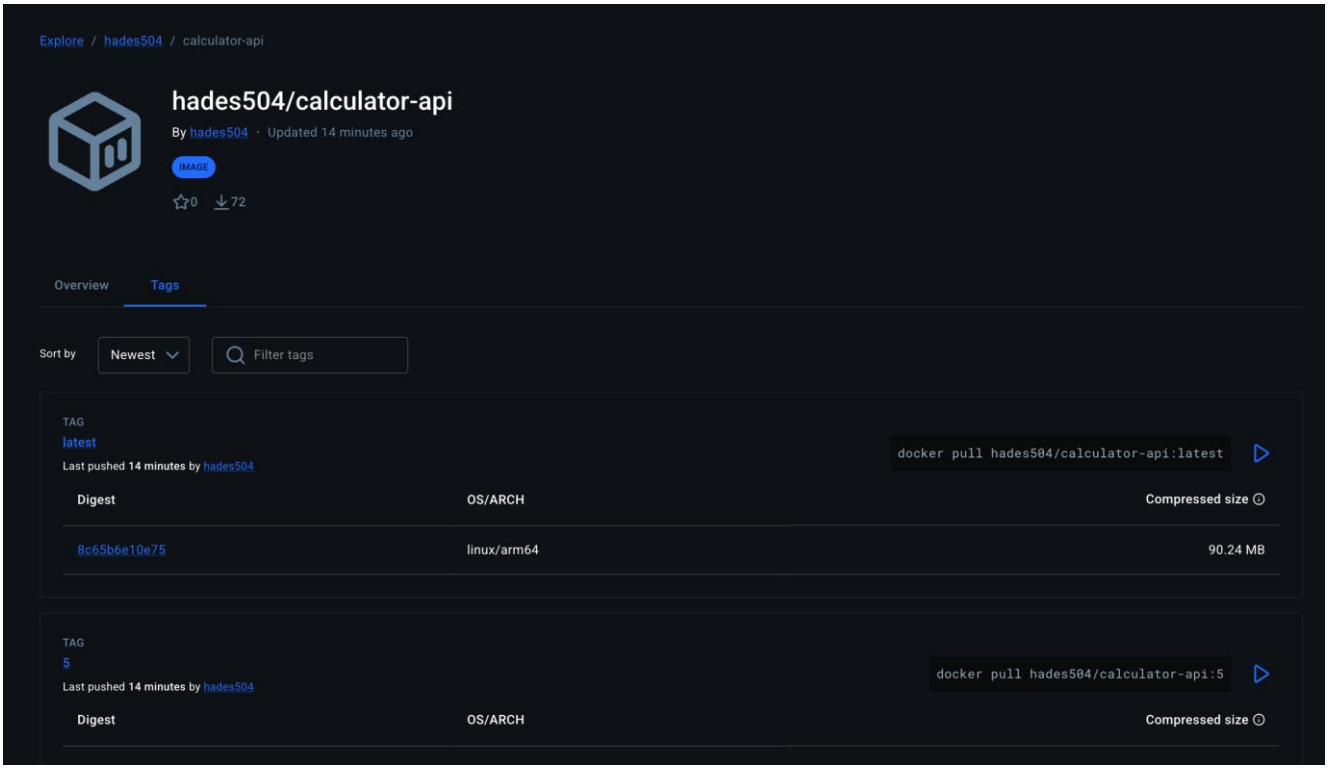


Figure A8: Docker Hub repository hades504/calculator-api with build-number tags

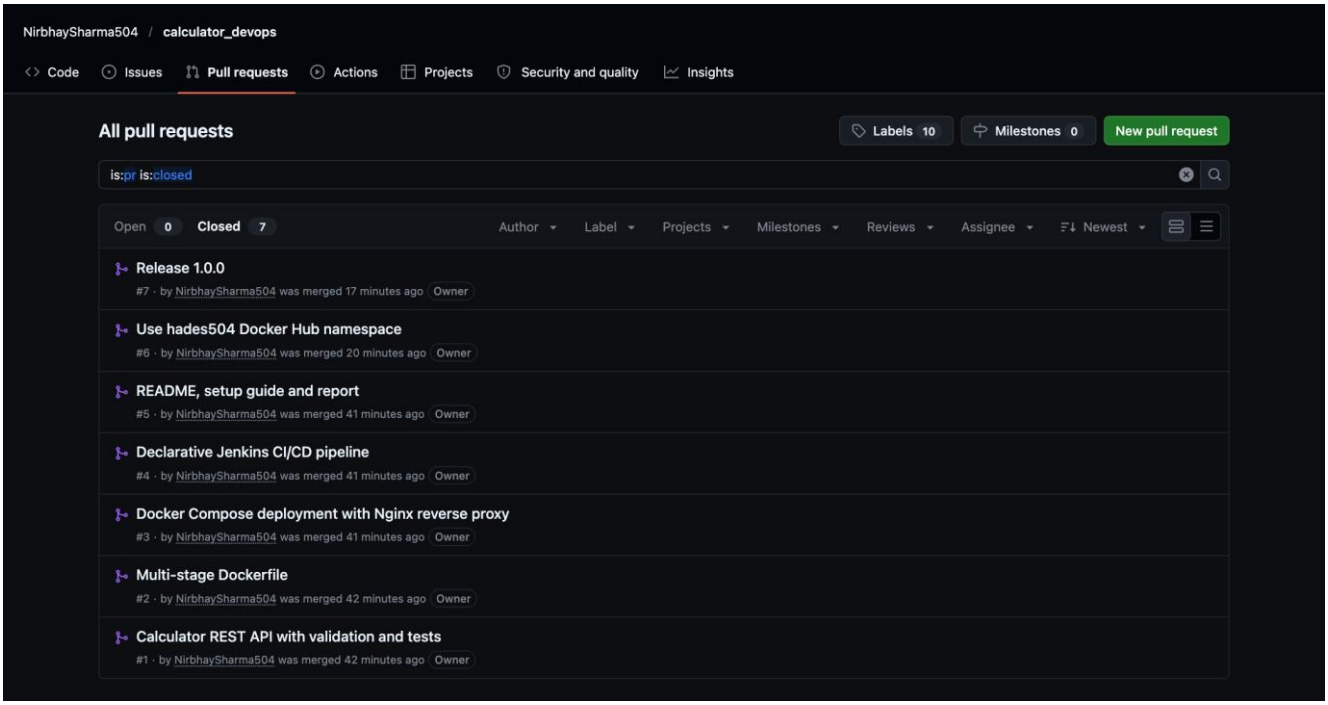


Figure A9: GitHub pull requests: 7 feature branches merged into main

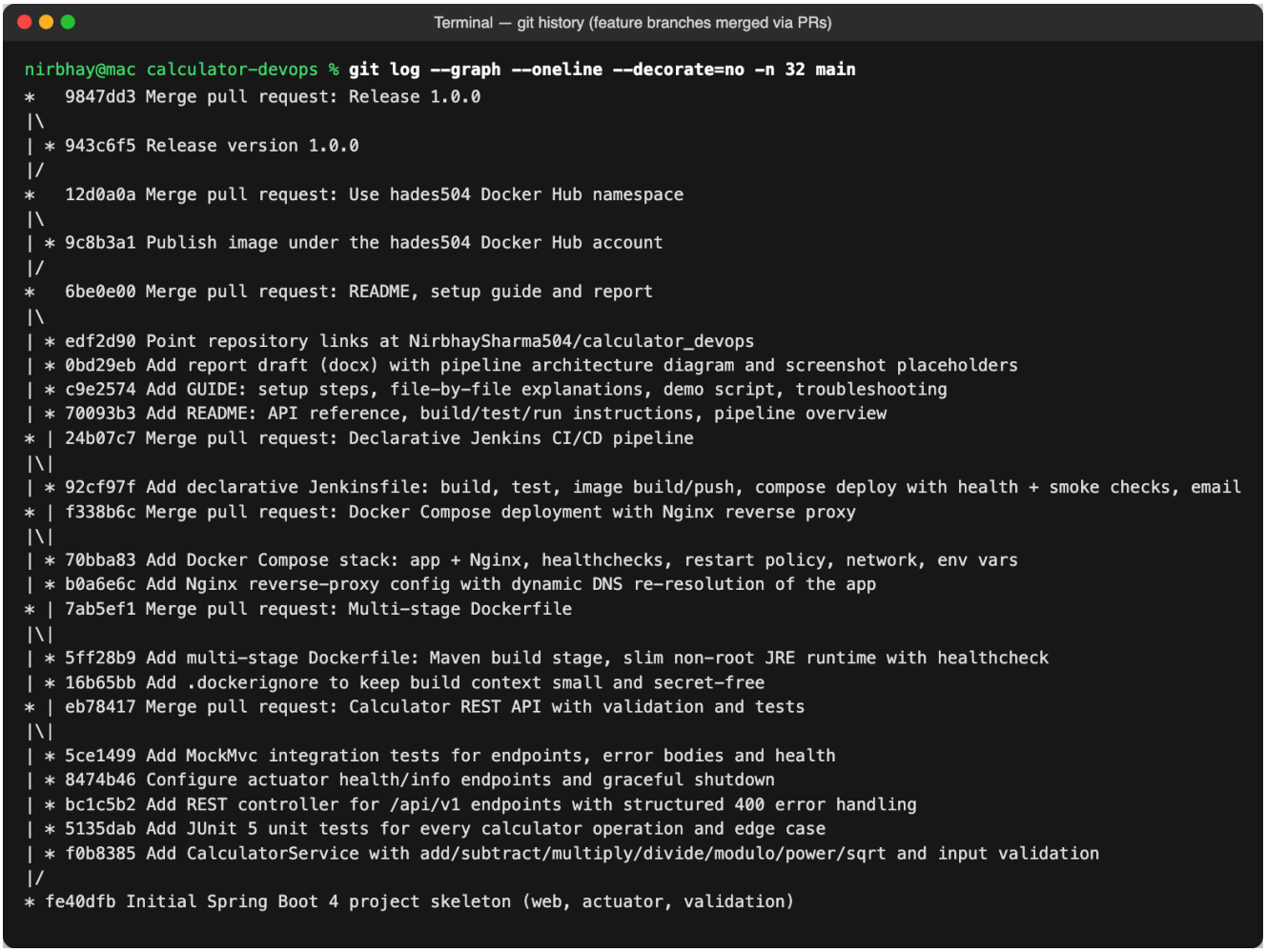


Figure A10: Commit history of main showing each feature branch merged

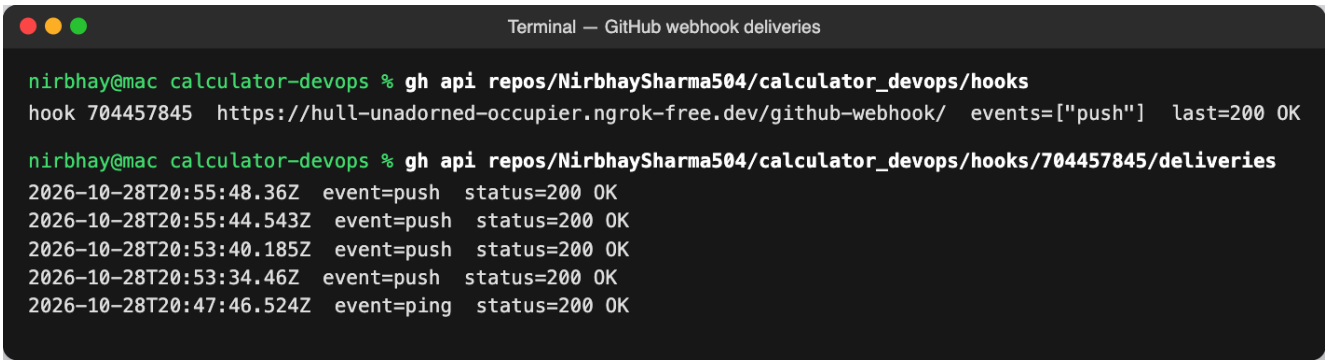


Figure A11: GitHub webhook to Jenkins (via ngrok): every delivery returned HTTP 200

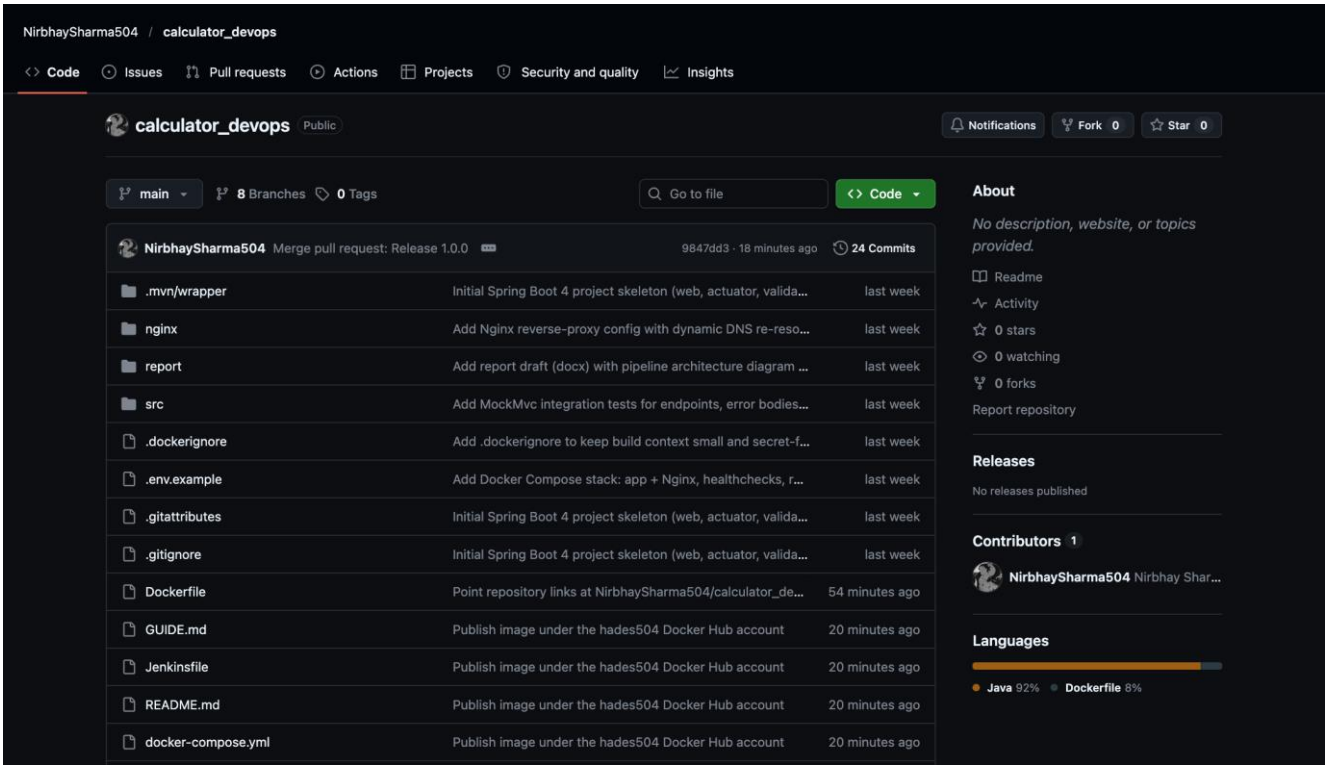


Figure A12: GitHub repository NirbhaySharma504/calculator_devops